

Intermediate (.env by hand) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Where this exercise is saved: `practice/env_config_practice.py`, under its `# Intermediate` section — the same file also holds the [Edge cases exercise](#), under an `# Edge cases` section. Run it with `cd practice && python env_config_practice.py`.

Used later by: the [Build Task](#)'s `load_config()` in `practice/build_task/config.py` — it ships the short `load_dotenv() + os.getenv()` version you land on here, re-written (not imported) so it can return a typed `Config` object.

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

A `.env` file is just lines of text like `KEY=value`. To read it yourself, open the file, look at each line one at a time, and split each line into the part before the `=` and the part after.

Skip any line that's empty.

Things to use:

- `open(path)` then a `for` loop to go through each line.
- `.strip()` to clean up each line.
- `.split("=", 1)` to break it into key and value.
- A dictionary (`{}`) to store the results.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Parsing this by hand touches a few real Python skills: reading a file line by line, string splitting with a limit, and building a dictionary as you go.

`for line in open(path):` iterates over a file one line at a time without loading the whole thing into memory first. `line.strip()` removes the trailing newline character each line comes with. `line.split("=", 1)` splits on the first `=` only (the second argument, `1`, is the max number of splits) — this matters because a value could itself contain an `=` character, and you don't want to split on that one too.

The exact pieces:

- `open(path)` used in a `for line in open(path):` loop — Python handles closing the file for you at the end of iteration in this pattern (though a `with open(path) as f:` block is the more correct, explicit way — worth knowing for later).
- `line.strip()` — removes leading/trailing whitespace, including the newline `\n` at the end of each line.
- Skip logic: `if not line or "=" not in line: continue` — handles blank lines and any malformed lines safely.
- `key, value = line.split("=", 1)` — unpacks the 2-item list `split()` returns directly into two variables.
- For the library comparison: `from dotenv import load_dotenv; load_dotenv(); os.getenv("KEY")`.

Difference between Basic and Intermediate: Basic describes the plain idea and lists the tools for the tidy, happy-path case. Intermediate shows the real Python syntax for that same tidy case — this is the depth the exercise's Solution is written at.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
make an empty dictionary called env_vars

open the file, go through it line by line:
    remove extra spaces from the line
    if the line is empty, skip to the next one
    if there's no "=" in the line, skip it too
    split the line at the first "=" into key and value
    save env_vars[key] = value

return env_vars
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# practice/env_config_practice.py – Intermediate section
def parse_env_by_hand(path):
    env_vars = {}
    for line in open(path):
        line = line.strip()
        if line == "" or "=" not in line:
            continue
        parts = line.split("=", 1)
        key = parts[0]
        value = parts[1]
        env_vars[key] = value
    return env_vars
```

Expected output if you run just this (nothing calls the function yet): nothing — defining a function doesn't run it. Add a call like `print(parse_env_by_hand(".env"))` below it, against a `.env` file containing `OPENAI_API_KEY=sk-test123`, to see it print `{'OPENAI_API_KEY': 'sk-test123'}`.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
function parse_env_by_hand(path) -> dict:
    env_vars = {}
    for line in open(path):
        line = line.strip()
        if not line or "=" not in line:
            continue
        key, value = line.split("=", 1)
        env_vars[key] = value
    return env_vars
```

```
compare against:
    from dotenv import load_dotenv
    load_dotenv()
    api_key = os.getenv("OPENAI_API_KEY")
```

```
# practice/env_config_practice.py – Intermediate section
def parse_env_by_hand(path: str) -> dict:
    env_vars = {}
    for line in open(path):
        line = line.strip()
        if not line or "=" not in line:
            continue
        key, value = line.split("=", 1)
        env_vars[key] = value
    return env_vars
```

Now write the 2-line `python-dotenv` version yourself and compare both against the [Solution](#).

Difference between Basic and Intermediate: same underlying idea (open, split, collect) at 2 completeness levels, shown here both as pseudocode and as near-complete code — Basic's version only ever collects whatever key/value pairs it finds, Intermediate adds the type contract real Python expects while still just collecting and returning no matter what.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)